

Ejercicio 4 — Justificación de diseño y mediciones

Práctica 2: FreeRTOS

Sistemas de Tiempo Real — Ingeniería en Computación

Consigna: utilizando las herramientas de sincronización, implemente una aplicación multitarea que muestre constantemente en la terminal las secuencias A, B y C. Todas las tareas deben tener la misma prioridad.

1. El problema

El enunciado obliga a que las tres tareas tengan la misma prioridad, y eso deja al planificador fuera de juego como mecanismo de ordenamiento. Con igual prioridad y `configUSE_TIME_SLICING = 1`, FreeRTOS reparte la CPU en *round-robin* tick a tick: el orden en que aparecen los mensajes depende de en qué momento del tick se bloquea cada tarea, y no es reproducible.

Esta es la diferencia de fondo con el Ejercicio 2. Allí el orden salía de las prioridades, y bastaba cambiar un `#define`. Acá el orden tiene que salir de la sincronización, que es justamente lo que el ejercicio quiere que se use.

Hay además una dificultad que no aparece en el caso más simple: las secuencias B y C **repiten tareas** (2, 2, 3, 1 y 3, 3, 3, 1, 2). Una cadena fija del tipo “cada tarea despierta a la siguiente” resuelve la secuencia A, pero no las otras dos.

2. La solución: testigo circulante con tabla de secuencia

Se usa un **semáforo binario por tarea**. Cada tarea arranca bloqueada esperando el suyo; cuando lo recibe imprime, espera el retardo del paso y le entrega el semáforo a la que sigue. Es un testigo que circula: como hay uno solo, únicamente una tarea puede estar fuera del estado *Blocked* a la vez, y el orden queda garantizado por construcción.

Para resolver las repeticiones, el “quién sigue” no está cableado dentro de cada tarea sino en un arreglo que las tres consultan:

```
static const uint8_t seq[] = { 2, 2, 3, 1 };    /* secuencia B */
static volatile uint8_t idx = 0;

xSemaphoreTake(sem[yo], portMAX_DELAY);        /* espero mi turno */
Serial.print(...);                             /* imprimo */
vTaskDelay(PASO_TICKS);                        /* espero el paso */
idx = (idx + 1) % SEQ_LEN;
xSemaphoreGive(sem[ seq[idx] ]);                /* paso el testigo */
```

Si `seq[idx]` vuelve a ser la misma tarea, ésta se da el semáforo a sí misma y en la vuelta siguiente lo toma sin bloquearse. Con eso las tres secuencias salen del mismo código: lo único que cambia entre A, B y C es el contenido de `seq[]`.

Decisiones puntuales

- **Semáforo binario y no mutex.** Acá una tarea toma el semáforo y *otra* lo libera, que es el uso por eventos. Un mutex no serviría: tiene dueño y lo tiene que devolver la misma tarea que lo tomó.
- **No hace falta proteger la UART.** Como solo una tarea está desbloqueada a la vez, nunca hay dos compitiendo por imprimir. El mecanismo que ordena la secuencia ya serializa el acceso al recurso, así que el mutex del Ejercicio 2 acá sobra.
- **El retardo va antes de pasar el testigo.** Si fuera después, la tarea siguiente pasaría a *Ready* de inmediato y, al bloquearse la actual, arrancaría en el acto: los mensajes saldrían todos juntos y el retardo solo separaría a cada tarea de sí misma, no un paso del siguiente.
- **idx no necesita protección.** Solo la toca la tarea que tiene el testigo, y hay un único testigo, así que no hay acceso concurrente. Se declara `volatile` únicamente para poder inspeccionarla desde el depurador.
- **Prioridad 1 y no 0.** La prioridad 0 es la de la tarea *Idle*, que en esta librería es la que ejecuta `loop()`. Las tres tareas comparten la prioridad 1, que cumple el requisito del enunciado sin competir con *Idle*.

3. Base de tiempo del RTOS

Este es el punto que más condiciona las mediciones, y no es evidente a menos que uno vaya a mirar la configuración de la librería.

El tick no sale del cristal de 16 MHz. La librería Arduino_FreeRTOS usa el Watchdog Timer como fuente de tick, y el watchdog corre con el oscilador RC interno de 128 kHz:

Parámetro	Valor	De dónde sale
portUSE_WDTO	WDTO_15MS	FreeRTOSVariant.h
portTICK_PERIOD_MS	$2^{(0+4)} = 16$ ms	prescaler de 2048 sobre 128 kHz
configTICK_RATE_HZ	$128000 \gg 11 = 62$ Hz	truncado de 62,5 Hz

De ahí sale una trampa concreta. La macro `pdMS_TO_TICKS()` usa división entera, así que pedir 200 ms da:

$$\text{pdMS_TO_TICKS}(200) = \frac{200 \cdot 62}{1000} = 12 \text{ ticks} = 12 \cdot 16 \text{ ms} = 192 \text{ ms}$$

Es decir que el retardo real es 192 ms y no los 200 ms pedidos, un error del 4%. Por eso en el código los retardos se expresan **directamente en ticks**: el valor queda exacto y la cuenta cierra con lo medido.

Hay una segunda consecuencia, más de fondo para un sistema de tiempo real: el RC de 128 kHz tiene tolerancia propia y deriva con tensión y temperatura, cosa que un cristal no hace. El reloj del RTOS y el reloj real del sistema son fuentes distintas y se separan con el tiempo. Para hacerlo visible, el programa imprime los dos en paralelo:

- `xTaskGetTickCount()` — tiempo del RTOS, derivado del watchdog.
- `micros()` — tiempo de TIMER0, derivado del cristal de 16 MHz.

4. Tiempos esperados

Con `PASO_TICKS = 12` cada turno dura $12 \cdot 16 = 192$ ms, y el ciclo completo es $T_{\text{ciclo}} = \text{SEQ_LEN} \cdot 192$ ms:

	Secuencia	Pasos	Ciclo teórico
A	Tarea 1 - Tarea 3 - Tarea 2	3	576 ms
B	Tarea 2 - Tarea 2 - Tarea 3 - Tarea 1	4	768 ms
C	Tarea 3 - Tarea 3 - Tarea 3 - Tarea 1 - Tarea 2	5	960 ms

5. Mediciones

Las tres secuencias se ejecutaron sobre el `.elf` compilado, instrumentando la salida de la UART y los flancos de PB1, PB2 y PB3. Resultados:

Magnitud	Teórico	Medido	Observación
Duración de un turno	192 ms	191,89 ms	error < 0,06 %
Ciclo secuencia A	576 ms	576 ms	
Ciclo secuencia B	768 ms	768 ms	
Ciclo secuencia C	960 ms	960 ms	
Primer turno	192 ms	206,48 ms	arranque no alineado al tick
Latencia de handoff	—	107 μ s	give + cambio de contexto + take

Dos resultados merecen comentario.

El primer turno dura 206,5 ms y no 192 ms. La cadena arranca desde `setup()`, en un instante cualquiera dentro de un tick. Como `vTaskDelay()` es relativo y cuantiza al tick, el primer paso se come hasta un tick extra (16 ms) antes de entrar en régimen. A partir del segundo turno el valor se estabiliza en 191,89 ms. Es el tipo de error de fase que en una tarea periódica se corrige con `xTaskDelayUntil()`, que mide desde un instante absoluto y no acumula deriva.

La latencia de handoff es de 107 μ s, y se repitió con esa misma cifra en todos los pasos medidos. Es el hueco entre el flanco de bajada de una tarea y el de subida de la siguiente, o sea el costo de `xSemaphoreGive` + cambio de contexto + `xSemaphoreTake`. A 16 MHz son unos 1712 ciclos de CPU. Ese número es el sobrecosto que el RTOS agrega a cada paso de la secuencia, y es lo que habría que sumar al análisis si los plazos fueran ajustados.

Consumo de recursos

Recurso	Uso	Disponible
Flash	11.926 B (36,4 %)	32 kB
RAM estática (.data+.bss)	374 B (18,3 %)	2 kB
Stack por tarea (pico)	67 B de 192	quedan 125 B libres

El valor de stack sale de `uxTaskGetStackHighWaterMark()`, que el programa imprime al cerrar cada ciclo. El margen es holgado, pero conviene no bajar `STACK_TAREA` mucho más: `Serial.print()` de enteros arma el número en un buffer sobre la pila de la tarea.

6. Cómo reproducir las mediciones en Proteus

1. Cargar en el ATmega328P el archivo `.elf` en lugar del `.hex`. El `.elf` está compilado con `-Og -g3` e incluye información DWARF, lo que habilita breakpoints sobre el código fuente.
2. Para los tiempos conviene el **analizador lógico sobre PB1, PB2 y PB3** (D9, D10 y D11). Cada pin marca qué tarea tiene el testigo; el hueco entre el flanco de bajada de una y el de subida de la siguiente es la latencia de handoff. Con breakpoints se congela la simulación y se pierde justamente lo que se quiere medir.
3. Los dos puntos marcados con `>>> BREAKPOINT` en el código sirven para verificar la lógica, no los tiempos: el `xSemaphoreTake` se alcanza una vez por turno, y en el `xSemaphoreGive` se pueden inspeccionar `idx` y `seq[idx]`.
4. Con `IMPRIMIR_TIEMPOS = 0` la terminal muestra solo **Tarea N**, que es la salida que pide la consigna. Con el flag en 1 cada línea agrega el tick del RTOS y el intervalo medido por el cristal.

Aclaración sobre la marca de los pines. El pin en alto significa que esa tarea *tiene el testigo*, no que esté usando la CPU: durante casi todo el turno la tarea está en *Blocked* dentro del `vTaskDelay()`, y la CPU la tiene la tarea *Idle*.

7. Limitaciones conocidas

- **Piso para PASO_TICKS.** A 9600 8N1 cada carácter tarda $10/9600 = 1,0417$ ms. Una línea con los tiempos ocupa unos 40 caracteres, es decir ≈ 42 ms de UART. `Serial.print()` escribe en el buffer de TX (64 bytes) y vuelve enseguida, sin esperar a que salga el último bit, así que con turnos de 192 ms el buffer nunca se llena. Si se bajara `PASO_TICKS` por debajo de unos 3 ticks (48 ms) el buffer sí se llenaría y `Serial.print()` pasaría a ser bloqueante, contaminando la medición.
- **Cuantización al tick.** `vTaskDelay()` es relativo, así que el error por paso es de hasta un tick (16 ms). En una cadena larga conviene `xTaskDelayUntil()`.
- **Deriva del reloj del RTOS.** Los 16 ms del tick dependen del RC interno de 128 kHz, no del cristal. Las mediciones de `micros()` son las confiables en términos absolutos; las de `xTaskGetTickCount()` son las que ve el planificador.
- **Sobre el entorno de medición.** Los números de este informe se obtuvieron ejecutando el `.elf` en un simulador de AVR. Ese simulador no implementa el modo interrupción del watchdog del ATmega328P, que es la fuente de tick de la librería, por lo que el tick de 16 ms se inyectó externamente con el mismo período. El firmware no se modificó. Los valores conviene reconfirmarlos en Proteus, donde el tick lo genera el watchdog real.

Referencias: apuntes de la cátedra, *Clase — Concurrencia, Paralelismo y SOTR* (Sistemas de Tiempo Real, Ingeniería en Computación, 2026); documentación de FreeRTOS (*Semaphore / Mutex API, Task Control*); código fuente de la librería `Arduino_FreeRTOS` (`FreeRTOSVariant.h`, `FreeRTOSConfig.h`, `port.c`); hoja de datos del ATmega328P, capítulos de USART y de Watchdog Timer.